

git

Table of Contents

commands

concepts

examples

problems

workflow

Comandos de Git

Basicos

Comando	Descripcion
<code>git init</code>	Inicializa un repositorio nuevo
<code>git clone <url></code>	Clona un repositorio remoto
<code>git add <archivo></code>	Agrega archivo al staging area
<code>git add .</code>	Agrega todos los cambios al staging
<code>git commit -m "msg"</code>	Confirma los cambios del staging
<code>git status</code>	Muestra el estado del working directory
<code>git log --oneline</code>	Historial compacto de commits
<code>git log --graph --oneline</code>	Historial con grafo de ramas
<code>git diff</code>	Diferencias sin staging
<code>git diff --staged</code>	Diferencias en el staging
<code>git show <commit></code>	Muestra detalles de un commit

Ramas

Comando	Descripcion
<code>git branch</code>	Lista ramas locales
<code>git branch <nombre></code>	Crea una rama nueva
<code>git branch -d <nombre></code>	Elimina una rama (segura)
<code>git branch -D <nombre></code>	Elimina una rama (forzada)
<code>git branch -a</code>	Lista todas las ramas (locales + remotas)
<code>git checkout <rama></code>	Cambia a otra rama
<code>git switch <rama></code>	Cambia a otra rama (forma moderna)
<code>git switch -c <rama></code>	Crea y cambia a rama nueva
<code>git merge <rama></code>	Fusiona la rama indicada en la actual
<code>git rebase <rama></code>	Reaplica commits encima de otra rama

Stash

Comando	Descripcion
<code>git stash</code>	Guarda cambios sin committear
<code>git stash push -m "msg"</code>	Guarda con descripcion
<code>git stash list</code>	Lista todos los stashes
<code>git stash pop</code>	Aplica y elimina el ultimo stash
<code>git stash apply</code>	Aplica sin eliminar el stash
<code>git stash drop</code>	Elimina el ultimo stash
<code>git stash clear</code>	Elimina todos los stashes

Remotos

Comando	Descripcion
<code>git remote -v</code>	Lista remotos configurados
<code>git remote add origin <url></code>	Agrega un remote
<code>git fetch</code>	Descarga cambios sin merge
<code>git pull</code>	Fetch + merge automatico
<code>git pull --rebase</code>	Fetch + rebase automatico
<code>git push</code>	Sube commits al remote
<code>git push -u origin <rama></code>	Push y establece upstream
<code>git push --force</code>	Fuerza push (cuidado)

Deshacer cambios

Comando	Descripcion
<code>git reset HEAD~1</code>	Deshace ultimo commit (mixed, default)
<code>git reset --soft HEAD~1</code>	Deshace commit, mantiene cambios en staging
<code>git reset --hard HEAD~1</code>	Deshace commit, elimina todos los cambios
<code>git revert <commit></code>	Crea un commit nuevo que invierte el indicado
<code>git clean -fd</code>	Elimina archivos sin trackear

Avanzados

Comando	Descripcion
<code>git cherry-pick <commit></code>	Aplica un commit especifico en la rama actual
<code>git bisect start</code>	Inicia busqueda binaria de bug
<code>git bisect good</code>	Marca commit como bueno
<code>git bisect bad</code>	Marca commit como malo
<code>git reflog</code>	Historial de movimientos de HEAD
<code>git blame <archivo></code>	Muestra quien modifico cada linea
<code>git tag <nombre></code>	Crea un tag en el commit actual
<code>git tag -a <nombre> -m "msg"</code>	Crea tag con anotacion
<code>git rebase -i HEAD~n</code>	Rebase interactivo para reordenar/squash

Conceptos de Git

Qué es Git

Git es un sistema de control de versiones distribuido. Registra cambios en archivos a lo largo del tiempo para poder consultar qué cambió, quién lo hizo, cuándo, y volver a versiones anteriores.

- **Distribuido:** cada desarrollador tiene una copia completa del historial
- **Snapshots:** cada commit es una instantánea del estado del proyecto
- **Branching:** permite crear ramas para trabajar en features aisladas

Áreas de trabajo

TEXT

Working Directory → Staging Area → Repository → Remote
(archivos) (git add) (git commit) (git push)

- **Working Directory:** archivos en tu disco
- **Staging Area:** archivos preparados para el proximo commit
- **Repository:** historial completo de commits
- **Remote:** repositorio en GitHub/GitLab/etc.

Qué es un Commit

Un commit es un guardado del estado del proyecto en un momento dado. Cada commit tiene un hash único, un autor, un mensaje y apunta al commit anterior.

BASH

```
git add archivo.js          # preparar cambios
git commit -m "feat: descripcion"  # crear commit
```

HEAD

HEAD apunta al commit actual (la punta de la rama donde estas).

TEXT

```
HEAD~1 = un commit atras  
HEAD~2 = dos commits atras  
HEAD~n = n commits atras
```

Merge vs Rebase

	Merge	Rebase
Resultado	Crea un commit de merge	Reaplica commits encima
Historial	Conserva la forma real	Historial lineal
Conflictos	Se resuelven una vez	Puede requerir resolver en cada commit
Uso	Ramas compartidas / produccion	Ramas personales / feature branches

Regla de oro: No hacer rebase en ramas que otros estan usando.

Qué es un Merge

Un merge es el proceso de combinar los cambios de dos ramas en una sola. Cuando terminas de trabajar en una feature branch, haces merge para integrar ese trabajo en main.

BASH

```
git switch main  
git merge feature/nueva-funcionalidad
```

Fast-forward Merge

Cuando la rama destino no tiene commits nuevos, Git solo mueve el puntero:

TEXT

```
Antes:  
main:    A → B → C  
feature:      C → D  
  
Merge (fast-forward):  
main:    A → B → C → D
```

Three-way Merge

Cuando ambas ramas tienen commits nuevos, Git crea un commit de merge:

TEXT

```
Antes:
main:   A → B → C → E
feature:      C → D

Merge:
main:     A → B → C → E → M (merge commit)
              └─┬─┘
                  D
```

Detached HEAD

Cuando `HEAD` apunta directamente a un commit en vez de a una rama. Puede pasar al hacer `git checkout <commit-hash>`.

TEXT

```
HEAD → commit C (no a una rama)

Solucion:
git switch -c nueva-rama  # crear rama desde ahi
git switch main           # volver a una rama
```

.gitignore

Archivo que le dice a Git que ignorar.

GITIGNORE

```
# Dependencias  
node_modules/
```

```
# Build  
dist/  
build/
```

```
# Env  
.env  
.env.local
```

```
# OS  
.DS_Store  
Thumbs.db
```

```
# IDE  
.vscode/  
.idea/
```

Ejemplos de Git

Crear repositorio y subir a GitHub

BASH

```
mkdir mi-proyecto && cd mi-proyecto
git init
git add .
git commit -m "feat: initial commit"
git remote add origin https://github.com/user/repo.git
git push -u origin main
```

Guardar trabajo a medio hacer

BASH

```
# Estas trabajando y necesitas cambiar de rama
git stash push -m "wip: avanzando en feature X"

# Cambiar a otra rama, hacer algo, volver
git switch main
# ... trabajo urgente ...

# Volver a tu trabajo
git switch feature/x
git stash pop
```

Rebase interactivo para limpiar commits

BASH

```
# Tienes 5 commits messy, quieres convertirlos en 1 o 2
git rebase -i HEAD~5

# Editor se abre:
pick abc1234 feat: add component
pick def5678 fix typo
pick ghi9012 fix typo again
pick jkl3456 feat: add styles
pick mno7890 fix: adjust spacing

# Cambia los ultimos 4 a "squash" o "fixup":
pick abc1234 feat: add component
squash def5678 fix typo
fixup ghi9012 fix typo again
squash jkl3456 feat: add styles
fixup mno7890 fix: adjust spacing
```

Cherry-pick un commit de otra rama

BASH

```
# Necesitas un commit especifico de otra rama
git log --oneline feature/otra # encontrar el hash
git cherry-pick abc1234
```

Buscar el commit que rompio algo (bisect)

BASH

```
git bisect start
git bisect bad # commit actual esta roto
git bisect good abc1234 # este commit funcionaba

# Git te lleva a un commit intermedio
# Probar si funciona:
git bisect good # o
git bisect bad

# Repetir hasta encontrar el commit
git bisect reset # volver al estado original
```

Problemas y soluciones

1. Undo ultimo commit (sin subir)

BASH

```
# Mantener cambios en staging (listos para recommittear)
git reset --soft HEAD~1

# Mantener cambios en working directory (sin staging)
git reset HEAD~1

# Eliminar todo (CUIDADO: no se puede deshacer)
git reset --hard HEAD~1
```

2. Undo commit que ya se pusheo

BASH

```
# Opcion segura: crear commit que invierte el cambio
git revert <commit-hash>
git push

# Opcion destructiva (no recomendado si otros tienen el commit)
git reset --hard HEAD~1
git push --force
```

3. Resolver conflictos de merge

BASH

```
git merge feature/rama
# CONFLICT en archivo.js

# 1. Abrir el archivo y buscar los conflictos:
<<<<<<< HEAD
    codigo de tu rama
=====
    codigo de la otra rama
>>>>>>> feature/rama

# 2. Editar: quedarte con lo que quieras, eliminar los markers

# 3. Marcar como resuelto
git add archivo.js

# 4. Completar el merge
git commit -m "merge: resolve conflicts"
```

4. Recuperar commit perdido (reflog)

BASH

```
# Ver historial de movimientos de HEAD
git reflog

# Output:
# abc1234 HEAD@{0}: reset: moving to HEAD~3
# def5678 HEAD@{1}: commit: feat: mi feature
# ghi9012 HEAD@{2}: commit: add styles

# Recuperar el commit
git cherry-pick def5678
# o
git checkout def5678
git switch -c rama-recuperada
```

5. Separar un commit en varios

BASH

```
# Reset el commit pero mantener los cambios
git reset --soft HEAD~1

# Ahora selecciona que archivos committear
git add archivo1.js
git commit -m "feat: parte 1"

git add archivo2.js
git commit -m "feat: parte 2"
```

6. Reordenar commits

BASH

```
# Rebase interactivo
git rebase -i HEAD~3

# En el editor, cambia el orden de las lineas:
pick abc1234 segundo commit
pick def5678 primer commit
pick ghi9012 tercer commit
```

7. Quitar archivo del historial completo

BASH

```
# Eliminar un archivo de todos los commits
git filter-branch --force --index-filter \
  "git rm --cached --ignore-unmatch archivo-secreto.env" \
  --prune-empty -- --all

# Limpiar referencias
git reflog expire --expire=now --all
git gc --prune=now --aggressive

# Agregar a .gitignore para que no vuelva
echo "archivo-secreto.env" >> .gitignore
git add .gitignore
git commit -m "chore: ignore sensitive file"
```

8. Fix detached HEAD

BASH

```
# Estas en detached HEAD y quieres guardar estos cambios
git switch -c rama-nueva # crear rama desde HEAD actual

# O volver a la rama anterior
git switch main
```

9. Saltar un commit en rebase

BASH

```
git rebase -i HEAD~5

# Cambiar "pick" a "skip" en el commit que quieres saltar:
pick abc1234 commit 1
skip def5678 commit 2 # este se salta
pick ghi9012 commit 3
```

10. Cambiar mensaje del ultimo commit

BASH

```
# Solo si no se pusheo
git commit --amend -m "nuevo mensaje"

# Si ya se pusheo (cuidado con force push)
git commit --amend -m "nuevo mensaje"
git push --force
```

11. Eliminar un archivo del staging (sin borrar del disco)

BASH

```
git reset HEAD archivo.js
```

12. Ver cambios sin committear

BASH

Todos los cambios

```
git diff
```

Solo los que estan en staging

```
git diff --staged
```

Cambios en un archivo especifico

```
git diff archivo.js
```

Resumen de cambios

```
git diff --stat
```

Flujo de trabajo

Feature Branch

BASH

```
# 1. Crear rama desde main
git switch main
git pull
git switch -c feature/nombre

# 2. Trabajar y commitear
git add .
git commit -m "feat: descripcion"

# 3. Push al remote
git push -u origin feature/nombre

# 4. Crear Pull Request / Merge Request

# 5. Despues del merge, limpiar
git switch main
git pull
git branch -d feature/nombre
git push origin --delete feature/nombre
```

Hotfix

BASH

```
# 1. Desde main, crear rama de hotfix
git switch main
git pull
git switch -c hotfix/bug-description

# 2. Fix y commit
git add .
git commit -m "fix: descripcion del bug"

# 3. Merge a main
git switch main
git merge hotfix/bug-description
git push

# 4. Limpiar
git branch -d hotfix/bug-description
```